

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

Name: K. Dedeeppya

Reg no: 192324252

1.Design a Machine Learning model to identify eligible beneficiaries for government welfare schemes.

1. Problem Statement

Government welfare schemes provide financial and social support to eligible citizens. Manual verification of applications can be time-consuming and may result in errors. A Machine Learning model can analyse applicant socio-economic information and predict whether an applicant is eligible for a welfare scheme.

2. Sustainable Development Goal

The proposed system supports SDG 1 by helping identify people who are eligible for welfare assistance and improving the delivery of government benefits.

3. Objective

The main objectives are:

- To analyse applicant socio-economic information.
- To identify important eligibility factors.
- To train a Machine Learning classification model.
- To predict whether an applicant is eligible or not eligible.

4. Background Study

Welfare eligibility may depend on annual income, family size, employment status, age, number of dependents and housing condition. Machine Learning can learn patterns from previously verified applications and use them to classify new applicants.

5. Learning Problem

This is a supervised classification problem.

- **Input:** Applicant socio-economic data
- **Target:** Eligible / Not Eligible
- **Algorithm:** Decision Tree
- **Learning Type:** Supervised Learning

6. Dataset

A sample dataset can be created containing applicant information.

Income	Family Size	Age	Dependents	Employment	Eligible
15000	5	45	3	No	Yes
60000	3	35	1	Yes	No
20000	6	52	4	No	Yes
55000	2	30	0	Yes	No
18000	4	60	3	No	Yes

7. Input Features and Target

Input Features:

- Annual Income
- Family Size
- Age
- Number of Dependents
- Employment Status

Target: Eligibility status

8. Data Preparation

The dataset is prepared by:

1. Checking missing values.
2. Removing duplicate records.
3. Converting categorical values into numerical values.
4. Selecting relevant features.
5. Dividing data into training and testing sets.

9. Algorithm – Decision Tree

Applicant Data



Income Check



Low Income?



Eligible



Final Result

10. Workflow

Applicant Dataset



Data Preprocessing



Feature Selection



Decision Tree



Test Model



Prediction



Eligible / Not Eligible

11. Python Implementation



```
[11] import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    "Income": [15000, 60000, 20000, 55000, 18000, 70000,
               12000, 45000, 16000, 30000],
    "FamilySize": [5, 3, 6, 2, 4, 2, 3, 5, 2],
    "Age": [45, 35, 52, 38, 68, 28, 50, 40, 55, 32],
    "Dependents": [1, 1, 4, 0, 3, 0, 3, 1, 3, 0],
    "Employment": [0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
    "Eligible": [1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)
X = df[["Income", "FamilySize", "Age",
        "Dependents", "Employment"]]
y = df["Eligible"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, prediction)
print("Predictions:", prediction)
print("Accuracy:", accuracy)
```

Predictions: [1 0 0]
Accuracy: 1.0

12. Advantages

- Faster application screening.
- Reduces manual workload.
- Can process many applications.
- Easy-to-understand Decision Tree.
- Supports welfare distribution.

13. Limitations

- Small or biased datasets may produce unreliable results.

- Personal data must be protected.
- Predictions should not completely replace human verification.
- Government eligibility rules may change.

14. Conclusion

The proposed Machine Learning system uses a Decision Tree to classify welfare applicants as eligible or not eligible. Pandas and NumPy help prepare and analyse the data, while Scikit-learn is used to train the model. The system can support faster and more consistent welfare beneficiary identification.

2. Develop a Machine Learning solution for crop disease detection using leaf images.

1. Problem Statement

Crop diseases can reduce agricultural production and cause economic losses to farmers. Early identification helps farmers take appropriate action. Machine Learning can classify crop leaves as healthy or diseased by analysing image-based features.

2. Sustainable Development Goal

SDG 2 – Zero Hunger

Early crop disease detection can reduce crop losses and improve food production, contributing to SDG 2.

3. Objective

- To represent leaf images using measurable features.
- To identify patterns associated with crop diseases.
- To train a classification model.
- To predict whether a leaf is healthy or diseased.

4. Background Study

Diseased leaves may show changes in colour, texture, spots and affected area. These characteristics can be converted into numerical features and given to a Machine Learning model.

5. Learning Problem

This is a supervised classification problem.

- **Input:** Leaf characteristics
- **Target:** Healthy / Diseased
- **Algorithm:** Decision Tree

6. Dataset

A simple feature-based dataset can contain:

Colour Intensity	Spot Area	Texture	Leaf Area	Class
180	5	20	95	Healthy
120	40	65	90	Diseased
175	8	22	92	Healthy
110	55	70	88	Diseased
185	4	18	96	Healthy

7. Features

- Colour intensity
- Disease spot area
- Texture value
- Leaf area

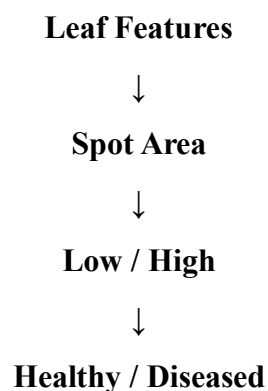
Target: Healthy / Diseased

8. Data Preparation

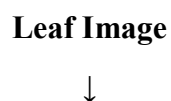
1. Collect leaf images.
2. Remove poor-quality images.
3. Extract numerical features.
4. Assign labels.
5. Check missing values.
6. Split the dataset into training and testing data.

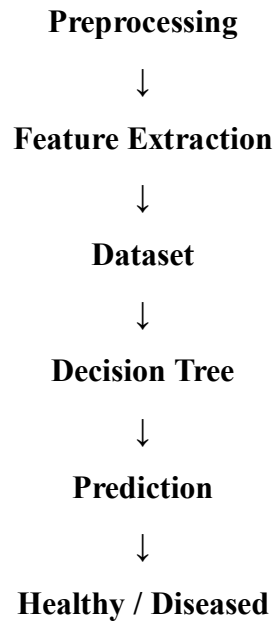
9. Algorithm

Decision Tree classifies the leaf based on feature values.

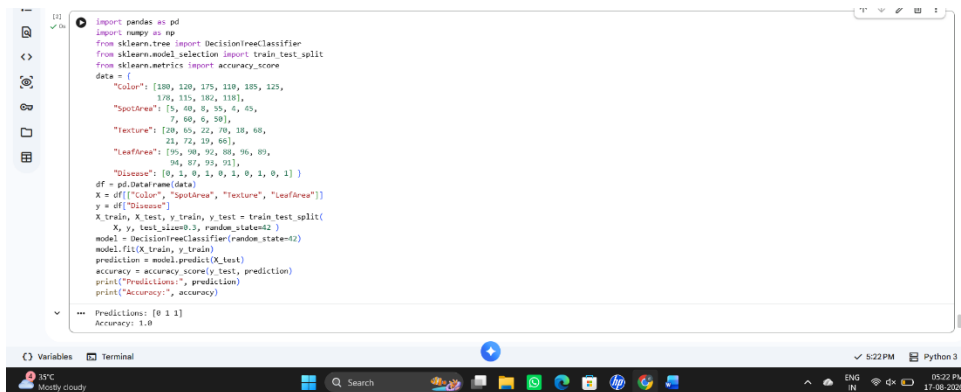


10. Workflow





11. Python Implementation



```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    "Color": [180, 120, 175, 110, 185, 125,
              170, 115, 182, 118],
    "SpotArea": [5, 60, 0, 35, 4, 65,
                 7, 60, 6, 50],
    "Texture": [20, 65, 22, 70, 18, 60,
               23, 72, 19, 66],
    "LeafArea": [95, 90, 92, 88, 96, 89,
                 94, 87, 93, 91],
    "Disease": [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)
X = df[["Color", "SpotArea", "Texture", "LeafArea"]]
y = df["Disease"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, prediction)
print("Predictions:", prediction)
print("Accuracy:", accuracy)

--- Predictions: [0 1 1]
Accuracy: 1.0
```

12. Advantages

- Early disease detection.
- Reduces crop losses.
- Saves inspection time.
- Supports farmers.
- Can be integrated into smart agriculture systems.

13. Limitations

- Simple features may not identify every disease.
- Lighting can affect image features.
- Similar diseases may have similar appearances.
- A larger real-world dataset is required.

14. Conclusion

The proposed system uses leaf characteristics and a Decision Tree to classify crop leaves as healthy or diseased. It demonstrates how Machine Learning can support early crop disease detection and contribute to sustainable agriculture.

3. Design a Machine Learning-based quality inspection system for manufacturing.

1. Problem Statement

Manufacturing industries need to identify defective products before they reach customers. Manual quality inspection can take time and may result in human errors. Machine Learning can analyse product measurements and predict whether a product is defective or non-defective.

2. Sustainable Development Goal

SDG 9 – Industry, Innovation and Infrastructure

The system supports SDG 9 by improving industrial automation, product quality and manufacturing efficiency.

3. Objective

- Analyse manufacturing data.
- Identify important quality parameters.
- Train a classification model.
- Predict product quality.
- Reduce defective products.

4. Background Study

Manufactured products can develop defects due to incorrect dimensions, abnormal temperature, pressure, weight or production conditions. Historical inspection data can be used to train a model that learns these patterns.

5. Learning Problem

This is a **supervised classification problem**.

- **Input:** Product and production parameters
- **Target:** Defective / Non-Defective
- **Algorithm:** Decision Tree

6. Dataset

Dimension	Weight	Temperature	Pressure	Defect
10.0	50	70	30	No
12.5	55	95	50	Yes
10.2	51	72	31	No

13.0	58	100	55	Yes
9.9	49	68	29	No

7. Input Features

- Product dimension
- Product weight
- Temperature
- Pressure

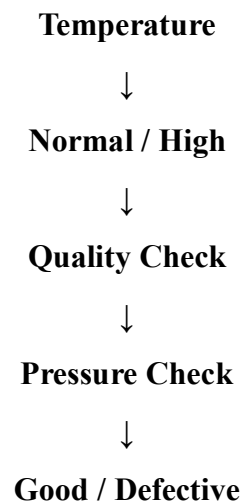
Target: Defective / Non-Defective

8. Data Preparation

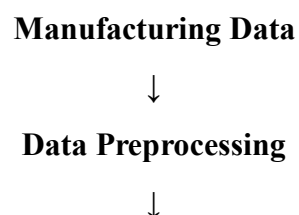
1. Collect production records.
2. Remove duplicate records.
3. Check missing values.
4. Select relevant features.
5. Convert target values into numerical form.
6. Divide data into training and testing sets.

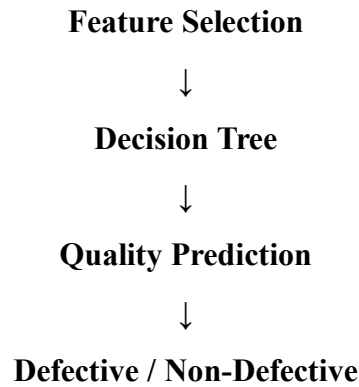
9. Algorithm

Decision Tree uses production conditions to make quality decisions.



10. Workflow





11. Python Implementation



```
[1] import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    "Dimension": [10.0, 12.5, 10.2, 13.0, 9.9,
                  12.8, 10.1, 13.2, 10.3, 12.7],
    "Weight": [50, 55, 51, 58, 49, 57, 50, 60, 52, 56],
    "Temperature": [70, 95, 72, 100, 68,
                   98, 71, 102, 73, 96],
    "Pressure": [30, 50, 31, 55, 29,
                 52, 30, 58, 32, 51],
    "Defect": [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)
X = df[["Dimension", "Weight", "Temperature", "Pressure"]]
y = df["Defect"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, prediction)
print("Predictions:", prediction)
print("Accuracy:", accuracy)
```

Predictions: [0 1 1]
Accuracy: 1.0

12. Advantages

- Faster quality inspection.
- Reduces manual errors.
- Improves product quality.
- Reduces production waste.
- Supports industrial automation.

13. Limitations

- Requires good-quality production data.
- Sensor errors can affect results.
- New defect types may not be recognized.
- Human verification may still be required.

14. Conclusion

The proposed Decision Tree model analyses manufacturing parameters and predicts whether products are defective. It can reduce manual inspection effort and improve production quality. The system demonstrates the application of Machine Learning in smart manufacturing.

4. Design an AI application for differently-abled people using Machine Learning.

1. Problem Statement

Differently-abled people may face difficulties in communication and accessing digital services. An AI-based assistive application can use Machine Learning to recognize commands from speech, gestures or other inputs and provide accessible responses.

2. Sustainable Development Goal

SDG 10 – Reduced Inequalities

The system promotes accessibility and reduces barriers faced by differently-abled people.

3. Objective

- Recognize user commands.
- Provide accessible responses.
- Support independent communication.
- Demonstrate Machine Learning classification.
- Improve accessibility.

4. Proposed Application

The proposed system is an AI-based assistive command recognition application. The system receives input features related to a user's command and classifies them into predefined commands.

For example:

- Help
- Stop
- Call
- Yes
- No

5. Learning Problem

This is a supervised classification problem.

- **Input:** User input features
- **Target:** Command class
- **Algorithm:** Decision Tree
- **Learning Type:** Supervised Learning

6. Dataset

A simple command dataset can be represented using numerical features.

Voice Feature	Gesture Feature	Movement	Command
80	10	20	Help
30	80	70	Stop
75	15	25	Help
25	85	75	Stop
60	20	30	Yes

7. Data Representation

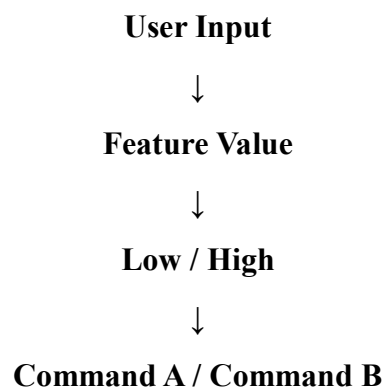
User inputs are converted into numerical features. The features can represent voice characteristics, gesture measurements or movement patterns.

8. Data Preparation

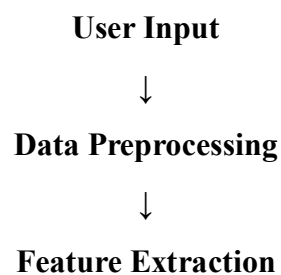
1. Collect labelled command data.
2. Remove incorrect records.
3. Handle missing values.
4. Extract important features.
5. Encode command labels.
6. Divide data into training and testing sets.

9. Algorithm

A Decision Tree classifies the input based on feature values.



10. Workflow

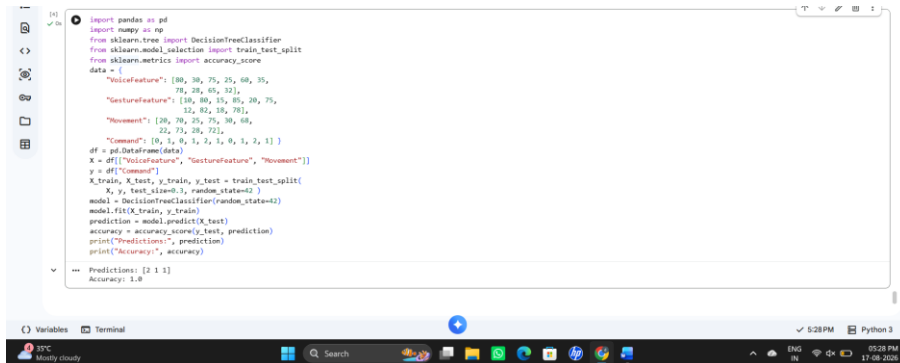


↓
Decision Tree

↓
Command Recognition

↓
Accessible Response

11. Python Implementation



```
[14]: import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    "VoiceFeature": [80, 90, 75, 25, 60, 35,
                    70, 20, 65, 30],
    "GestureFeature": [10, 80, 15, 85, 20, 75,
                     32, 62, 50, 70],
    "Movement": [20, 70, 25, 75, 30, 60,
                22, 73, 28, 72],
    "Command": [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)
X = df[["VoiceFeature", "GestureFeature", "Movement"]]
y = df["Command"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, prediction)
print("Predictions:", prediction)
print("Accuracy:", accuracy)
```

Predictions: [1 1]
Accuracy: 1.0

12. Advantages

- Improves accessibility.
- Supports independent communication.
- Reduces digital barriers.
- Provides quick responses.
- Can be customized.

13. Limitations

- Recognition errors may occur.
- Different users have different communication patterns.
- More training data is required for real-world use.
- User privacy must be protected.

14. Conclusion

The proposed AI application uses Machine Learning to recognize user commands and provide accessible responses. A Decision Tree provides a simple and understandable classification method. The application can support communication and reduce accessibility barriers.

5.Design a smart parking prediction system using Machine Learning.

1. Problem Statement

Finding available parking in crowded cities takes time and increases unnecessary vehicle movement. A Machine Learning system can analyse parking information and predict whether a parking area will be available or full.

2. Sustainable Development Goal

SDG 11 – Sustainable Cities and Communities

Smart parking improves the use of urban infrastructure and can reduce unnecessary traffic movement.

3. Objective

- Analyse historical parking data.
- Identify parking availability patterns.
- Train a prediction model.
- Predict available or full parking.
- Support smart city development.

4. Background Study

Parking availability depends on time, day, location, occupancy and nearby events. Machine Learning can learn these patterns from historical data and predict future parking conditions.

5. Learning Problem

This is a **supervised classification problem**.

- **Input:** Parking-related information
- **Target:** Available / Full
- **Algorithm:** Decision Tree

6. Dataset

Time	Day	Occupancy	Event	Available
9	Weekday	30	No	Yes
10	Weekday	85	Yes	No
11	Weekday	40	No	Yes
18	Weekday	90	Yes	No
15	Weekend	45	No	Yes

7. Input Features

- Time
- Day type
- Current occupancy
- Nearby event
- Historical parking usage

Target: Available / Full

8. Data Preparation

1. Collect parking records.
2. Remove duplicate records.
3. Handle missing values.
4. Convert categorical data into numerical form.
5. Select important features.
6. Divide the dataset into training and testing sets.

9. Algorithm

Decision Tree checks parking conditions and predicts availability.

Parking Occupancy



High / Low



Parking Status



Full / Available

10. Workflow

Parking Dataset



Data Preprocessing



Feature Selection



Decision Tree

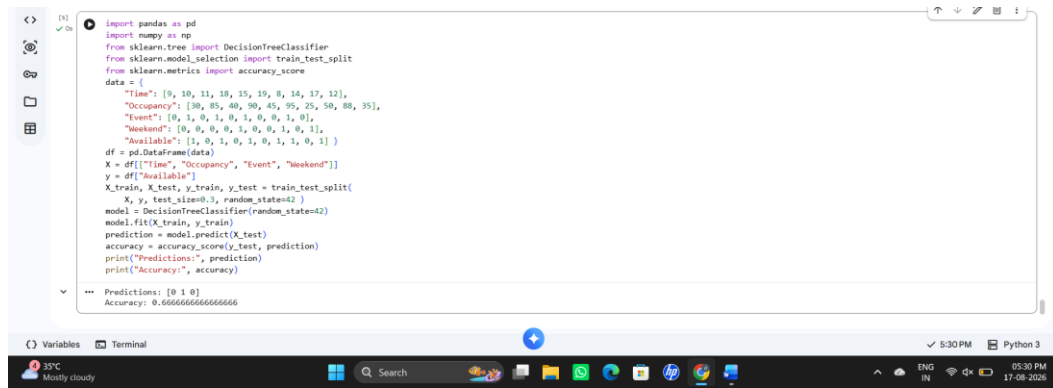


Parking Prediction



Available / Full

11. Python Implementation



```
[1] In [ ]: import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    "Time": [9, 10, 11, 10, 15, 19, 0, 14, 17, 12],
    "Occupancy": [30, 85, 40, 90, 45, 95, 25, 50, 88, 35],
    "Event": [0, 1, 0, 1, 0, 1, 0, 0, 0, 0],
    "Weekend": [0, 0, 0, 0, 1, 0, 0, 1, 0, 1],
    "Available": [1, 0, 1, 0, 1, 0, 1, 1, 0, 1]
}

df = pd.DataFrame(data)
X = df[["Time", "Occupancy", "Event", "Weekend"]]
y = df["Available"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, prediction)
print("Predictions:", prediction)
print("Accuracy:", accuracy)

*** Predictions: [0 1 0]
Accuracy: 0.6666666666666666
```

12. Advantages

- Reduces parking search time.
- Reduces unnecessary vehicle movement.
- Helps reduce traffic congestion.
- Improves parking utilization.
- Supports smart city planning.

13. Limitations

- Accuracy depends on data quality.
- Sudden events may change parking demand.
- Sensor errors may affect predictions.
- New parking areas require additional data.

14. Conclusion

The proposed Machine Learning system uses a Decision Tree to predict parking availability. Pandas and NumPy support data processing, while Scikit-learn is used for model development. The system can reduce parking search time and contribute to smarter and more sustainable cities.